SOFTWARE TEST CASE DOCUMENTATION

Smart Waste Sorter System

AI / Computer Vision Waste Classification

Project name	Smart Waste Sorter System
Document title	Software Test Case Documentation
Project type	AI / Computer Vision Waste Classification
Main technologies	Python, OpenCV, YOLO (**to be confirmed** against the project source code)
Prepared by	[Student Name]
Class	[Class Name]
Institution	[School Name]
Document version	1.0
Preparation date	26 October 2026
Document status	Draft – test cases planned, **no tests executed yet**

Document Structure

Page	Section	Content type
1	Cover Page and Document Structure	Project and document details
2	Introduction and Project Overview	Purpose, objectives, workflow, users
3	Testing Scope, Environment, and Strategy	Scope, test types, requirements
4–7	Detailed Test Cases (TC-001 to TC-030)	Test case tables, categories A–E
8	Test Data and Execution Procedures	Test data, step-by-step procedure
9	Test Results, Defect Reporting, Evaluation	Summary, defect template
10	Conclusion and Recommendations	Checklist, references

Important: The project ZIP file was not received when this document was prepared. All project-specific details (libraries, model, waste categories, features) are therefore marked **Not verified** and must be confirmed against the real project before submission.

1. Introduction and Project Overview

1.1 Purpose of the System

The Smart Waste Sorter System is an AI-based application that uses a camera and a computer-vision model to recognise waste items and tell the user how to sort them (for example, which bin to use). Its purpose is to make waste sorting faster, easier, and more accurate.

1.2 Problem the System Solves

Many people place waste in the wrong bin because they are unsure about the material or the sorting rules. Mixed waste reduces recycling quality and increases landfill. The system helps by giving an instant, consistent recommendation based on what the camera sees.

1.3 Main Objectives

- Capture live images or video frames from a camera.
- Detect or classify waste items using a YOLO model (**Not verified**).
- Show the waste category and detection confidence to the user.
- Give a clear sorting recommendation for each category.
- Provide voice guidance so users can sort without reading the screen (**Not verified**).
- Optionally keep detection history and statistics (**Not verified**).

1.4 High-Level System Workflow

The expected workflow is shown below. Each stage must be confirmed against the actual source code.

Camera Input →	**Image Processing (OpenCV)** →	**YOLO Detection / Classification** →	**Waste Category** →	**Sorting Recommendation** →	**Voice Guidance / UI Output**

Stages marked "Not verified" until the code review is complete. Optional outputs (history, statistics) are included only if implemented.

1.5 Intended Users

- Students and teachers using the system in a classroom or school demonstration.
- General users who need guidance on how to sort household or school waste.
- Project evaluators who review the system's functions and quality.

1.6 Importance of Software Testing

Testing shows whether the system works as intended and finds errors before users do. For an AI system, testing is especially important because wrong predictions lead to wrong sorting advice. Testing also checks that the camera, interface, and voice features work together and fail safely when something goes wrong.

1.7 Scope of This Document

This document defines the **test plan and 30 planned test cases** for the Smart Waste Sorter System. It does **not** contain executed results or accuracy measurements. The student must run the tests, record real results, and update the status of each test case.

2. Testing Scope, Environment, and Strategy

2.1 Features Included and Excluded

Included in testing	Excluded from testing
Camera start, live feed, and failure handling Image/frame processing Waste detection and classification (YOLO) Confidence display and low-confidence handling Sorting recommendations Voice guidance (if implemented) User interface display History and statistics (if implemented) Missing-file handling and safe shutdown	Training the YOLO model or measuring official model accuracy (mAP) Physical bins, motors, or sensors (not part of this plan unless implemented) Network, cloud, or mobile features (if any) Security and penetration testing Large-scale load testing Any feature confirmed as **Not implemented**

2.2 Test Strategy

Test type	What will be checked	Test cases
Functional testing	Each feature gives the expected output for given input	TC-006 to TC-021
Integration testing	Camera → model → recommendation → voice/UI work together	TC-005, TC-014, TC-025
User interface testing	Results are visible, readable, and updated correctly	TC-002, TC-025, TC-028
Camera and image input	Camera starts, shows feed, fails safely	TC-001 to TC-005
AI detection/classification	Correct category, confidence, unknown/low-confidence handling	TC-006 to TC-014
Voice guidance testing	Correct speech, no repeated spam, safe failure	TC-020 to TC-024
Error handling/reliability	Missing model, camera loss, TTS failure, safe exit	TC-003, 004, 024, 029, 030
Basic performance	Observe smoothness of feed and response time (no invented targets)	TC-002, TC-005, TC-014

2.3 Hardware Requirements (to confirm)

Item	Requirement
Computer	Laptop/PC able to run Python and YOLO inference (**Requires confirmation**: CPU/GPU, RAM)
Camera	Built-in or USB webcam supported by OpenCV
Audio output	Speakers or headphones (needed for voice tests)

2.4 Software Requirements (to confirm)

Software / library	Version	Status
Python	Requires confirmation	Check with python --version
OpenCV (opencv-python)	Requires confirmation	Check with pip show opencv-python
YOLO framework (e.g., Ultralytics)	Requires confirmation	Check with pip show ultralytics (**Not verified**)
Text-to-speech library (e.g., pyttsx3)	Requires confirmation	**Not verified** – confirm in the code
UI library (e.g., Tkinter)	Requires confirmation	**Not verified** – confirm in the code
Operating system	Requires confirmation	Windows / macOS / Linux

2.5 Test Environment and Setup

1. Copy the project to a folder and open it in a terminal or IDE.
2. Create a virtual environment and install dependencies (e.g., pip install -r requirements.txt, if the file exists).
3. Check that the YOLO model file (e.g., a .pt file) and any configuration files are present.
4. Connect and test the camera and speakers, then start the application and note the start command.

3. Detailed Test Cases

Default values: Actual Result = “Not executed”, Status = “Not Run”. Allowed statuses: Not Run, Pass, Fail, Blocked, Not Applicable. Items marked “Not verified” must be confirmed in the code first.

ID	Title / Module	Objective / Preconditions	Test steps / Test data	Expected result	Actual / Status / Priority
A. Camera and Image Input Testing					
TC-001	**Start the camera successfully** Module: Camera module	**Objective:** Confirm the app opens the default camera. **Pre:** Webcam connected; dependencies installed.	1. Connect the webcam. 2. Run the main script (filename to confirm). 3. Wait up to 10 seconds. **Data:** Camera index 0 (default)	Camera opens with no error; a live video window/panel appears; app does not crash.	**Actual:** Not executed **Status:** Not Run **Priority:** High
TC-002	**Display the camera feed correctly** Module: Camera / UI	**Objective:** Verify the live feed is smooth and correct. **Pre:** TC-001 passes.	1. Start the app. 2. Wave a hand in front of the camera. 3. Watch the feed for 30 seconds. **Data:** Hand movement	Feed updates continuously, follows real movement, has normal colours, and does not freeze.	**Actual:** Not executed **Status:** Not Run **Priority:** High
TC-003	**Handle camera access failure** Module: Camera / Error handling	**Objective:** Check behaviour when the camera is busy. **Pre:** Camera used by another app.	1. Open a video-call or camera app. 2. Start the Smart Waste Sorter. 3. Observe the screen and terminal. **Data:** Camera in use	A clear error message is shown or logged; the app does not crash or hang.	**Actual:** Not executed **Status:** Not Run **Priority:** High
TC-004	**Handle an unavailable or disconnected camera** Module: Camera / Error handling	**Objective:** Check safe handling of a missing camera. **Pre:** App can start with camera.	1. Start the app with the USB camera running. 2. Unplug the camera. 3. Wait 10 seconds. 4. Restart the app with no camera. **Data:** Camera unplugged	A “camera not found” message appears or the app exits safely; no unhandled error trace.	**Actual:** Not executed **Status:** Not Run **Priority:** High
TC-005	**Process images or video frames correctly** Module: Image processing	**Objective:** Confirm frames reach the model and results are drawn. **Pre:** Camera works.	1. Start the app. 2. Show one bottle to the camera. 3. Check for a box and label on the frame. 4. Note any lag or frame drops. **Data:** One plastic bottle	Frames are processed continuously; annotated output is displayed; no crash.	**Actual:** Not executed **Status:** Not Run **Priority:** High
B. AI Waste Detection and Classification Testing					
TC-006	**Detect a supported plastic item** Module: AI detection	**Objective:** Verify plastic is detected. **Pre:** Model loaded; good light.	1. Start the app. 2. Hold a plastic bottle 30–50 cm away. 3. Keep it still for 3 seconds. 4. Read the label on screen. **Data:** Clear plastic water bottle	Object is boxed and labelled as plastic (or the project’s equivalent class).	**Actual:** Not executed **Status:** Not Run **Priority:** High
TC-007	**Detect a supported paper item** Module: AI detection	**Objective:** Verify paper is detected. **Pre:** Model loaded.	1. Start the app. 2. Hold a sheet of paper or cardboard in view. 3. Keep still for 3 seconds. 4. Read the label. **Data:** A4 paper / cardboard	Object is labelled as paper (or cardboard class if used).	**Actual:** Not executed **Status:** Not Run **Priority:** High

ID	Title / Module	Objective / Preconditions	Test steps / Test data	Expected result	Actual / Status / Priority
TC-008	**Detect a supported metal or can item** Module: AI detection	**Objective:** Verify metal/can is detected. **Pre:** Model loaded.	1. Start the app. 2. Hold an empty drink can in view. 3. Keep still for 3 seconds. 4. Read the label. **Data:** Empty aluminium can	Object is labelled as metal/can.	**Actual:** Not executed **Status:** Not Run **Priority:** High
TC-009	**Detect a supported glass item (if supported)** Module: AI detection	**Objective:** Verify glass detection. **Pre:** Glass class supported (**Not verified**).	1. Start the app. 2. Hold a glass bottle or jar in view. 3. Keep still for 3 seconds. 4. Read the label. **Data:** Empty glass bottle	Labelled as glass. If glass is not supported, mark **Not Applicable** with reason.	**Actual:** Not executed **Status:** Not Run **Priority:** Medium
TC-010	**Detect an organic waste item (if supported)** Module: AI detection	**Objective:** Verify organic waste detection. **Pre:** Organic class supported (**Not verified**).	1. Start the app. 2. Hold a banana peel or apple core in view. 3. Keep still for 3 seconds. 4. Read the label. **Data:** Banana peel	Labelled as organic/food waste. If unsupported, mark **Not Applicable**.	**Actual:** Not executed **Status:** Not Run **Priority:** Medium
TC-011	**Handle an unsupported or unknown object** Module: AI detection	**Objective:** Check unknown objects are not misclassified. **Pre:** Model loaded.	1. Start the app. 2. Show a pen, phone, or keys. 3. Watch for 10 seconds. 4. Record any label shown. **Data:** Pen, phone, keys	No waste label with high confidence; “unknown” or no detection; no wrong bin shown.	**Actual:** Not executed **Status:** Not Run **Priority:** High
TC-012	**Display detection confidence correctly** Module: AI detection / UI	**Objective:** Check confidence value is shown and valid. **Pre:** TC-006 passes.	1. Show a plastic bottle. 2. Read the confidence value. 3. Compare with the value in the terminal/log (if any). **Data:** Plastic bottle	Confidence is shown, within 0–100% (or 0–1), and matches the model output.	**Actual:** Not executed **Status:** Not Run **Priority:** Medium
TC-013	**Handle low-confidence predictions** Module: AI detection	**Objective:** Check behaviour below the threshold. **Pre:** Threshold value known (**Requires confirmation**).	1. Show a partly hidden or blurry object. 2. Dim the room light. 3. Observe label, confidence, and message. **Data:** Blurry/dim object	Predictions below the threshold are not accepted as final; a message or no result is shown.	**Actual:** Not executed **Status:** Not Run **Priority:** High
TC-014	**Test detection stability across frames** Module: AI detection / Integration	**Objective:** Check the label does not flicker. **Pre:** Stabilization implemented (**Not verified**).	1. Hold a bottle still for 10 seconds. 2. Count label changes. 3. Move it slowly and count again. **Data:** Plastic bottle	Label stays the same while the object is still; no rapid flickering between classes.	**Actual:** Not executed **Status:** Not Run **Priority:** Medium
C. Sorting Recommendation Testing					
TC-015	**Correct recommendation for plastic** Module: Sorting advice	**Objective:** Verify the plastic sorting advice. **Pre:** TC-006 passes.	1. Show a plastic bottle. 2. Read the recommendation. 3. Compare it with the project’s sorting rules. **Data:** Plastic bottle	Recommendation matches the plastic rule (e.g., plastic/recycling bin – confirm).	**Actual:** Not executed **Status:** Not Run **Priority:** High
TC-016	**Correct recommendation for paper** Module: Sorting advice	**Objective:** Verify the paper sorting advice. **Pre:** TC-007 passes.	1. Show a sheet of paper. 2. Read the recommendation. 3. Compare with the sorting rules. **Data:** A4 paper	Recommendation matches the paper rule (e.g., paper bin – confirm).	**Actual:** Not executed **Status:** Not Run **Priority:** High

ID	Title / Module	Objective / Preconditions	Test steps / Test data	Expected result	Actual / Status / Priority
TC-017	**Correct recommendation for metal or cans** Module: Sorting advice	**Objective:** Verify the metal sorting advice. **Pre:** TC-008 passes.	1. Show an empty can. 2. Read the recommendation. 3. Compare with the sorting rules. **Data:** Aluminium can	Recommendation matches the metal rule (e.g., metal/recycling bin – confirm).	**Actual:** Not executed **Status:** Not Run **Priority:** High
TC-018	**Correct recommendation for other supported categories** Module: Sorting advice	**Objective:** Verify glass/organic/other advice. **Pre:** Category supported (**Not verified**).	1. Show a glass or organic item. 2. Read the recommendation. 3. Compare with the sorting rules. **Data:** Glass bottle / banana peel	Recommendation matches the rule for that category. **Not Applicable** if no other categories exist.	**Actual:** Not executed **Status:** Not Run **Priority:** Medium
TC-019	**Handle an unknown category safely** Module: Sorting advice / Errors	**Objective:** Ensure no wrong or empty advice is given. **Pre:** App running.	1. Show an unsupported object. 2. Read the recommendation area. 3. Check the terminal for errors. **Data:** Pen or phone	Safe message (e.g., "unknown"/no advice) is shown; no crash or blank error.	**Actual:** Not executed **Status:** Not Run **Priority:** High
D. Voice Guidance Testing (if implemented – Not verified)					
TC-020	**Speak the detected waste category** Module: Voice guidance	**Objective:** Verify speech names the category. **Pre:** Speakers on; voice supported.	1. Set the volume to medium. 2. Show a plastic bottle. 3. Listen to the audio. **Data:** Plastic bottle	Voice says the correct category, clearly, matching the on-screen label.	**Actual:** Not executed **Status:** Not Run **Priority:** Medium
TC-021	**Speak the recommended bin or sorting instruction** Module: Voice guidance	**Objective:** Verify speech gives the bin advice. **Pre:** TC-020 passes.	1. Show a can. 2. Listen to the audio. 3. Compare with the on-screen recommendation. **Data:** Aluminium can	Voice gives the correct sorting instruction matching the screen.	**Actual:** Not executed **Status:** Not Run **Priority:** Medium
TC-022	**Announce recyclability or organic status** Module: Voice guidance	**Objective:** Check recyclability announcements. **Pre:** Feature supported (**Not verified**).	1. Show a plastic item, then a banana peel. 2. Listen for recyclable/organic wording. **Data:** Bottle; banana peel	Correct recyclable/organic status is spoken. **Not Applicable** if not implemented.	**Actual:** Not executed **Status:** Not Run **Priority:** Low
TC-023	**Avoid repeated announcements** Module: Voice guidance	**Objective:** Check the same object is not announced every frame. **Pre:** Voice works.	1. Hold one bottle in view for 15 seconds. 2. Count the announcements. **Data:** Plastic bottle (still)	Object announced once (or after a cooldown – value to confirm); no continuous repeating.	**Actual:** Not executed **Status:** Not Run **Priority:** High
TC-024	**Handle a voice or text-to-speech failure** Module: Voice / Error handling	**Objective:** Check the app survives speech failure. **Pre:** Voice implemented.	1. Mute or disable the audio device/TTS engine. 2. Show an object. 3. Observe detection and messages. **Data:** Audio disabled	No crash; detection and UI continue; error is logged or shown.	**Actual:** Not executed **Status:** Not Run **Priority:** Medium
E. User Interface, History, and Reliability Testing					
TC-025	**Display the detection result in the interface** Module: User interface	**Objective:** Verify all results are visible. **Pre:** App running.	1. Show a plastic bottle. 2. Check category, confidence, and recommendation. 3. Check text size and position. **Data:** Plastic bottle	Category, confidence, and recommendation are visible, readable, and update with the object.	**Actual:** Not executed **Status:** Not Run **Priority:** High

ID	Title / Module	Objective / Preconditions	Test steps / Test data	Expected result	Actual / Status / Priority
TC-026	**Save detection history (if implemented)** Module: History	**Objective:** Check detections are saved. **Pre:** History feature supported (**Not verified**).	1. Detect 3 different objects. 2. Open the history screen or file. 3. Count the entries. **Data:** 3 objects	3 entries are saved. **Not Applicable** if no history feature.	**Actual:** Not executed **Status:** Not Run **Priority:** Medium
TC-027	**Record category, timestamp, confidence** Module: History	**Objective:** Check history fields are correct. **Pre:** TC-026 passes.	1. Detect a bottle. 2. Open the history entry. 3. Compare with the screen and system clock. **Data:** Plastic bottle	Category, timestamp, and confidence match the detection.	**Actual:** Not executed **Status:** Not Run **Priority:** Medium
TC-028	**Display statistics correctly (if implemented)** Module: Statistics	**Objective:** Check statistics counts. **Pre:** Statistics supported (**Not verified**).	1. Detect 2 plastic items and 1 paper item. 2. Open the statistics view. **Data:** 2 plastic, 1 paper	Plastic = 2, paper = 1, total = 3. **Not Applicable** if no statistics.	**Actual:** Not executed **Status:** Not Run **Priority:** Low
TC-029	**Handle missing model or configuration files** Module: Error handling	**Objective:** Check safe failure without the model. **Pre:** Backup of the model file made.	1. Rename the model file temporarily. 2. Start the app. 3. Read the message. 4. Restore the file name. **Data:** Missing model file	Clear “model not found” message; app does not start detection or crash unclearly.	**Actual:** Not executed **Status:** Not Run **Priority:** High
TC-030	**Close the application safely** Module: Reliability	**Objective:** Check clean shutdown. **Pre:** App running.	1. Close using the exit key/button. 2. Check the camera light. 3. Check that no Python process remains. **Data:** Normal exit	Window closes, camera is released, no process remains, data saved (if any).	**Actual:** Not executed **Status:** Not Run **Priority:** Medium

4. Test Data and Test Execution Procedures

4.1 Test Data Required

Data group	Examples	Notes
Plastic objects	Water bottle, plastic cup, food container, plastic bag	Use clean, empty items; try clear and coloured plastic
Paper objects	A4 paper, cardboard box, newspaper, paper cup	Try flat and crumpled paper
Metal cans	Aluminium drink can, tin food can	Try shiny and dented cans
Glass / organic	Glass bottle or jar; banana peel, apple core	Use **only if supported** by the project
Unknown / background	Pen, phone, keys, hand, empty table, wall	Should not be classified as waste
Lighting	Bright room, normal room, dim room, backlight	Record the condition for every test
Distance and angle	20 cm, 50 cm, 100 cm; front, side, top view	Note the distance at which detection fails
Low-confidence / ambiguous	Blurry, partly hidden, or mixed-material items	Used for TC-011 and TC-013
Failure scenarios	Camera unplugged or busy; model file renamed; audio muted	Used for TC-003, 004, 024, 029

4.2 Step-by-Step Execution Procedure

1. **Prepare the environment:** set up the computer, camera, speakers, and a plain background.
2. **Install the dependencies:** create a virtual environment and install the required libraries.
3. **Confirm files:** check that the model file and configuration files exist in the project folder.
4. **Start the application:** run the main script and note any start-up messages.
5. **Prepare the test input:** choose the object or scenario listed in the test case.
6. **Perform the test steps:** follow the numbered steps exactly, in order.
7. **Record the actual result:** write what really happened, including the label and confidence.
8. **Compare results:** check the actual result against the expected result.
9. **Assign the status:** Pass, Fail, Blocked, or Not Applicable (with reason).
10. **Record evidence:** save screenshots, error messages, and log files; link them to the test case ID.

4.3 Important Testing Advice

- Use **several different real objects** for each category, not just one example. One correct result does not prove the model is reliable.
- Repeat each AI test at least a few times in different lighting, distances, and angles, and record every result honestly.
- If a feature does not exist in your project, mark the test **Not Applicable** and write the reason. Do not mark it Pass.
- Never change an expected result to match a wrong actual result. Report it as a defect instead.

Tester's log (fill in): Tester name: _____ Date tested: _____ Computer/OS: _____ Camera model: _____

5. Test Results, Defect Reporting, and Evaluation

5.1 Test Execution Summary (unconfirmed)

Measure	Count	Note
Total test cases	30	Planned
Passed test cases	Unconfirmed	Fill in after testing
Failed test cases	Unconfirmed	Fill in after testing
Blocked test cases	Unconfirmed	Fill in after testing
Not-run test cases	30 (default)	Update as tests are executed
Not-applicable test cases	Unconfirmed	Depends on which features exist

*Planned test cases are **not** completed tests. No pass rate or accuracy is claimed in this document.*

5.2 Defect Report Template

Field	Entry
Defect ID	DEF-001
Related test case ID	TC-___
Defect title	
Steps to reproduce	1. 2. 3.
Expected result	
Actual result	
Severity	Critical / Major / Minor / Cosmetic
Priority	High / Medium / Low
Status	New / Open / Fixed / Retested / Closed
Evidence / screenshot reference	File name or screenshot number

5.3 How to Evaluate Failures

Area	Evaluation guidance
AI classification	Record the object, shown label, confidence, lighting, and distance. A wrong label, missed object, or false detection on an unknown object is a **Fail**. Check whether the issue repeats before reporting.
Camera handling	A crash, freeze, blank feed, or missing error message when the camera fails is a **Fail**. A clear message with safe recovery is a **Pass**.
Voice guidance	Wrong wording, no sound when expected, continuous repeating, or a crash on audio failure is a **Fail**. Check volume and device settings first (could be **Blocked**).
User interface	Hidden, cut-off, unreadable, or outdated results are a **Fail**. Also check that text matches the voice output.

5.4 Observations and Potential Defects

Code-review findings: none – the project source code was not available for review. **Confirmed defects (by execution):** none – no tests have been executed.

Potential risks to check (general, **not confirmed**): wrong labels under poor lighting; flickering results; repeated voice announcements; missing handling for absent camera or model file.

6. Conclusion and Recommendations

6.1 Summary

Purpose: testing checks that the Smart Waste Sorter System detects waste correctly, gives the right sorting advice, and handles errors safely. **Modules covered:** camera input, image processing, AI detection, sorting recommendation, voice guidance, user interface, history/statistics, and reliability. **Expected benefits:** early discovery of defects, more reliable classification, and a better user experience. **Real results matter:** only honest, recorded results show the true quality of the system.

6.2 Recommended Improvements (to consider after testing)

- Show a clear “unknown object” message instead of a wrong category.
- Add a confidence threshold and result stabilization if not present.
- Limit voice announcements with a short cooldown.
- Show friendly messages for camera or model errors.
- Add test images covering different lighting, distances, and angles.

6.3 Future Testing Opportunities

- Measure accuracy with a larger, labelled test image set.
- Test on different computers and camera models.
- Conduct user-acceptance testing with classmates.
- Long-run stability testing (30 minutes or more).

6.4 Final Remarks

Good software quality means correct results, safe error handling, and clear communication to users. Careful testing helps make the system dependable and ready for demonstration.

6.5 Final Testing and Submission Checklist

Done	Task
[ ]	Project source code reviewed and all “Not verified” items confirmed
[ ]	Libraries and versions recorded in Section 2.4
[ ]	Unsupported features marked Not Applicable with reasons
[ ]	All test cases executed; actual results and statuses updated
[ ]	Defects recorded; screenshots saved and referenced
[ ]	Summary table in Section 5.1 updated with real numbers
[ ]	Student name, class, school, and date completed on the cover page
[ ]	Document exported to PDF and checked for cut-off text

6.6 References (recommended official documentation – to consult)

- Python documentation – https://docs.python.org/3/
- OpenCV documentation – https://docs.opencv.org/
- Ultralytics YOLO documentation – https://docs.ultralytics.com/ (if Ultralytics YOLO is used)

These sources are suggested for reference and have not been cited as consulted in the preparation of this document.